DEVELOPER DOCUMENTATION

Developer Integration Guide

Headless Billing API & Embed SDK

Version 1.0 · March 2026

© 2026 WHMS. Confidential.

Table of Contents

Introduction
Architecture Overview
Quick Start — React SDK
Script Tag Integration
Authentication & API Keys
Public API Reference
JavaScript Fetch Examples
SDK Components
SDK Hooks
End-to-End Integration Flow
Security Best Practices
SDK Package Structure
Use Case — Hosting Company
Troubleshooting

© 2026 WHMS. Confidential.

Introduction

The WHMS Billing Engine is a **headless, API-first** billing platform that handles the full subscription lifecycle — from plan discovery to payment collection and invoice generation. It is designed to integrate cleanly into any existing website or application without requiring you to adopt a monolithic control panel.

What the system provides

CAPABILITY	DESCRIPTION
Subscription Billing	Recurring billing with configurable cycles (monthly, annually, etc.)
Client Management	Register and authenticate end-users via API
Invoicing	Itemised invoices with tax, discounts, and payment tracking
Service Provisioning	Attach plans to services with pricing, setup fees, and add-ons
Embeddable UI	Drop-in React components and a plain-script widget bundle

Why use this approach?

- **Works with any website** — static sites, WordPress, Next.js, plain HTML — anything with a `<script>` tag or a `npm install`
- **No platform lock-in** — your frontend stays exactly as it is; the billing layer is an external API
- **Modern API-first architecture** — every action is a plain HTTP call authenticated by a scoped API key
- **Ship in minutes** — one `<script>` tag is all you need to show a live pricing table

© 2026 WHMS. Confidential.

Architecture Overview

The integration is a clean three-layer stack. Your website never touches the database directly — it communicates only through the public API layer.

```
┌──────────────────────────────────────────────────────┐
│                   Your Website / App                 │
│                                                      │
│   ┌────────────────┐     ┌──────────────────────┐   │
│   │  React SDK     │     │  <script> Widget     │   │
│   │  (npm package) │     │  (dist/widget.js)    │   │
│   └────────┬───────┘     └──────────┬───────────┘   │
└────────────┼─────────────────────────┼───────────────┘
             │  HTTPS + x-api-key      │
             ▼                         ▼
┌──────────────────────────────────────────────────────┐
│               Public API  /public/v1/                │
│                                                      │
│   GET  /plans          POST /clients                 │
│   POST /orders         POST /auth/login              │
│   GET  /invoices/:id   GET  /auth/me                 │
│                                                      │
│   ── apiKeyGuard ── requireScope ── publicClientAuth ── │
└──────────────────────────┬───────────────────────────┘
                           │
            ┌──────────────▼──────────────┐
            │        Billing Engine       │
            │                             │
            │  OrderService  InvoiceService │
            │  AuthService   PlanService  │
            │  ProvisioningService        │
            └──────────────┬──────────────┘
                           │
            ┌──────────────▼──────────────┐
            │        PostgreSQL           │
            │   (Prisma ORM)              │
            └─────────────────────────────┘
```

Request flow

```
Browser  ──[1. API key]──▶  Public API
         ◀─[2. Plans]──────  PlanService
         ──[3. Register]──▶  AuthService  ──▶  DB
         ──[4. Login]─────▶  AuthService  ──▶  JWT issued
         ──[5. Order]─────▶  OrderService ──▶  DB + Provisioning
         ──[6. Invoice]───▶  InvoiceService ──▶ DB
```

Quick Start — React SDK

The fastest way to embed billing into a React application.

1. Install

```
npm install @whms/billing-sdk
# peer dependencies (skip if already installed)
npm install react react-dom
```

2. Add a pricing table

```
import { PricingTable } from "@whms/billing-sdk";

export default function PricingPage() {
  return (
    <PricingTable
      apiKey="pk_test_xxxxxxxxxxxx"
      baseUrl="https://api.example.com"
      onSelect={(plan, pricing) => {
        // plan   → { id, name, summary, ... }
        // pricing → { id, billingCycle, price, currency, ... }
        console.log("Selected:", plan.name, pricing.billingCycle);
      }}
    />
  );
}
```

3. Add a login widget

```
import { LoginWidget } from "@whms/billing-sdk";

export default function AccountPage() {
  return (
    <LoginWidget
      apiKey="pk_test_xxxxxxxxxxxx"
      baseUrl="https://api.example.com"
      onLogin={({ user, accessToken }) => {
        // store accessToken for subsequent order / invoice requests
        sessionStorage.setItem("client_token", accessToken);
      }}
    />
  );
}
```

4. Show an order form

```
import { OrderForm } from "@whms/billing-sdk";

export default function CheckoutPage({ planId, pricingId, serviceId }) {
  return (
    <OrderForm
      apiKey="pk_test_xxxxxxxxxxxx"
      baseUrl="https://api.example.com"
      serviceId={serviceId}
      planId={planId}
      pricingId={pricingId}
      onSuccess={({ order, costBreakdown, accessToken }) => {
        console.log("Order created:", order.id);
      }}
    />
  );
}
```

> **Next.js note:** All SDK components use React hooks and must run in the browser. In the App Router, add `"use client"` at the top of any file that imports SDK components.

Script Tag Integration

No build tool, no npm — drop two tags into any HTML page.

Pricing Table

```
<!— 1. Load the widget bundle once —>
<script src="https://cdn.example.com/widget.js"></script>

<!— 2. Place a container anywhere on the page —>
<div id="pricing-table"></div>

<!— 3. Initialise —>
<script>
  BillingWidget.renderPricingTable({
    elementId: "pricing-table",
    apiKey:    "pk_test_xxxxxxxxxxxx",
    baseUrl:   "https://api.example.com",
    onSelect:  function(plan, pricing) {
      console.log(plan.name, pricing.price, pricing.currency);
    }
  });
</script>
```

Login widget

```
<div id="login-box"></div>

<script>
  BillingWidget.renderLoginWidget({
    elementId: "login-box",
    apiKey:    "pk_test_xxxxxxxxxxxx",
    baseUrl:   "https://api.example.com",
    onLogin:   function(result) {
      window.__clientToken = result.accessToken;
    }
  });
</script>
```

Invoice viewer

```
<div id="invoice"></div>

<script>
  BillingWidget.renderInvoiceViewer({
    elementId:   "invoice",
    apiKey:      "pk_test_xxxxxxxxxxxx",
    baseUrl:     "https://api.example.com",
    invoiceId:   "inv_abc123",
    clientToken: window.__clientToken   // JWT from login
  });
</script>
```

Available render methods

METHOD	DESCRIPTION
`BillingWidget.renderPricingTable(opts)`	Plans grid with cycle selector
`BillingWidget.renderOrderForm(opts)`	Registration + checkout flow
`BillingWidget.renderLoginWidget(opts)`	Email/password login
`BillingWidget.renderInvoiceViewer(opts)`	Invoice detail with line items

Each method returns `{ unmount }` — call `unmount()` to tear down the widget.

© 2026 WHMS. Confidential.

Authentication & API Keys

Key types

The system uses two categories of key. Keep them strictly separated.

KEY	PREFIX	USED WHERE	EXPOSES
Public key	`pk_`	Frontend, widget bundle, SDK	Plan data, login, order creation
Secret key	`sk_`	Backend server only	Admin operations, full billing access

```
pk_test_a1b2c3d4e5f6...   ← safe to include in client-side code
sk_test_z9y8x7w6v5u4...   ← NEVER in frontend code
```

Creating an API key

Keys are created through the admin portal or the management API.

```
curl -X POST https://api.example.com/api/auth/apikeys \
  -H "Authorization: Bearer <admin_token>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "My Website Widget",
    "scopes": [
      "plans.read",
      "clients.create",
      "auth.login",
      "orders.create",
      "invoices.read"
    ],
    "expiresInDays": 365
  }'
```

Response:

© 2026 WHMS. Confidential.

```
{
  "rawKey": "pk_live_xxxxxxxxxxxxxxxxxxxxx",
  "apiKeyId": "key_abc123",
  "scopes": ["plans.read","clients.create","auth.login","orders.create","invoices.read"]
}
```

> The raw key is shown **once**. Store it immediately — it cannot be retrieved again.

Available scopes

SCOPE	GRANTS ACCESS TO
`plans.read`	`GET /public/v1/plans`
`clients.create`	`POST /public/v1/clients`
`auth.login`	`POST /public/v1/auth/login` and `GET /public/v1/auth/me`
`orders.create`	`POST /public/v1/orders`
`invoices.read`	`GET /public/v1/invoices/:id`

A key with **no scopes** is treated as unrestricted — useful during development.

Client authentication

Once an end-user logs in, they receive a short-lived JWT (`accessToken`). Pass this token in subsequent requests via the `x-client-token` header. The server validates ownership before returning order or invoice data.

```
x-api-key: pk_test_xxx          ← identifies your integration
x-client-token: eyJhbGci...     ← identifies the logged-in customer
```

© 2026 WHMS. Confidential.

Public API Reference

Base URL: `https://api.example.com/public/v1`

All endpoints require the `x-api-key` header (or `Authorization: Bearer <key>`).

GET /plans

Returns all active plans, optionally filtered by service.

Query parameters

PARAMETER	TYPE	DESCRIPTION
`serviceId`	string (optional)	Filter plans to a specific service

Request

```
curl https://api.example.com/public/v1/plans \
  -H "x-api-key: pk_test_xxxxxxxxxxxx"
```

Response

```
{
  "success": true,
  "plans": [
    {
      "id": "plan_abc",
      "name": "Starter",
      "summary": "Perfect for small projects",
      "active": true,
      "position": 0,
      "pricing": [
        {
          "id": "price_xyz",
          "billingCycle": "monthly",
          "price": 9.99,
          "setupFee": 0,
          "currency": "USD",
          "active": true
        }
      ],
      "service": { "id": "svc_123", "name": "Web Hosting" }
    }
  ]
}
```

GET /plans/:planId

Returns a single plan by ID.

```
curl https://api.example.com/public/v1/plans/plan_abc \
  -H "x-api-key: pk_test_xxxxxxxxxxxx"
```

POST /clients

Registers a new client account.

Request body

FIELD	TYPE	REQUIRED
`email`	string	Yes
`password`	string (min 8 chars)	Yes

```
curl -X POST https://api.example.com/public/v1/clients \
  -H "x-api-key: pk_test_xxxxxxxxxxxx" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "securepass123"
  }'
```

Response `201 Created`

```
{
  "success": true,
  "client": {
    "id": "usr_abc123",
    "email": "user@example.com",
    "emailVerified": false
  }
}
```

POST /auth/login

Authenticates a client and returns JWT tokens.

```
curl -X POST https://api.example.com/public/v1/auth/login \
  -H "x-api-key: pk_test_xxxxxxxxxxxx" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "securepass123"
  }'
```

Response

```
{
  "success": true,
  "user": { "id": "usr_abc123", "email": "user@example.com" },
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiJ9..."
}
```

GET /auth/me

Returns the currently authenticated client. Requires `Authorization: Bearer <accessToken>`.

```
curl https://api.example.com/public/v1/auth/me \
  -H "x-api-key: pk_test_xxxxxxxxxxxx" \
  -H "Authorization: Bearer eyJhbGci..."
```

POST /orders

Creates an order for the authenticated client.

Headers

```
x-api-key: pk_test_xxxxxxxxxxxx
x-client-token: eyJhbGci...        ← required
Content-Type: application/json
```

Request body

FIELD	TYPE	REQUIRED
`serviceId`	string	Yes
`planId`	string	Yes
`pricingId`	string	Yes
`billingCycles`	integer	No (default: 1)
`quantity`	integer	No (default: 1)
`addons`	string[]	No

© 2026 WHMS. Confidential.

```
curl -X POST https://api.example.com/public/v1/orders \
  -H "x-api-key: pk_test_xxxxxxxxxxxx" \
  -H "x-client-token: eyJhbGci..." \
  -H "Content-Type: application/json" \
  -d '{
    "serviceId": "svc_123",
    "planId":    "plan_abc",
    "pricingId": "price_xyz",
    "billingCycles": 12
  }'
```

Response `201 Created`

```
{
  "success": true,
  "order": {
    "id": "ord_xyz789",
    "status": "pending",
    "createdAt": "2026-03-23T10:00:00.000Z"
  },
  "costBreakdown": {
    "baseCost": 119.88,
    "setupFee": 0,
    "total": 119.88,
    "currency": "USD"
  }
}
```

GET /invoices/:id

Returns a single invoice. The client token must belong to the invoice owner.

```
curl https://api.example.com/public/v1/invoices/inv_abc123 \
  -H "x-api-key: pk_test_xxxxxxxxxxxx" \
  -H "x-client-token: eyJhbGci..."
```

Response

```
{
  "success": true,
  "invoice": {
    "id": "inv_abc123",
    "invoiceNumber": "INV-2026-00042",
    "status": "unpaid",
    "currency": "USD",
    "subtotal": 119.88,
    "taxAmount": 0,
    "discountAmount": 0,
    "totalAmount": 119.88,
    "amountDue": 119.88,
    "dueDate": "2026-04-22T00:00:00.000Z",
    "issuedAt": "2026-03-23T10:00:00.000Z",
    "lineItems": [
      {
        "id": "li_001",
        "description": "Starter — monthly × 12",
        "quantity": 12,
        "unitPrice": 9.99,
        "total": 119.88,
        "taxRate": 0,
        "taxAmount": 0
      }
    ]
  }
}
```

Error responses

All errors follow the same shape:

```
{ "error": "Human-readable error message" }
```

HTTP STATUS	MEANING
`400`	Validation error — check your request body
`401`	Missing or invalid API key / client token
`403`	Insufficient scope or ownership mismatch
`404`	Resource not found
`409`	Conflict — e.g. email already registered
`500`	Internal server error

© 2026 WHMS. Confidential.

JavaScript Fetch Examples

Use these patterns in any environment that supports the Fetch API (browser, Node 18+, Deno, Cloudflare Workers).

Helper wrapper

```
const BASE    = "https://api.example.com/public/v1";
const API_KEY = "pk_test_xxxxxxxxxxxx";

async function billingFetch(path, options = {}) {
  const { clientToken, ...init } = options;

  const headers = {
    "Content-Type": "application/json",
    "x-api-key": API_KEY,
    ...init.headers,
  };

  if (clientToken) headers["x-client-token"] = clientToken;

  const res  = await fetch(`${BASE}${path}`, { ...init, headers });
  const data = await res.json();

  if (!res.ok) throw new Error(data.error || `HTTP ${res.status}`);
  return data;
}
```

Fetch plans

```
const { plans } = await billingFetch("/plans");
```

Register + login

```
// Register (ignore duplicate email errors)
try {
  await billingFetch("/clients", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
} catch (err) {
  if (!err.message.includes("already registered")) throw err;
}

// Login
const { accessToken, user } = await billingFetch("/auth/login", {
  method: "POST",
  body: JSON.stringify({ email, password }),
});
```

Create order

```
const { order, costBreakdown } = await billingFetch("/orders", {
  method: "POST",
  clientToken: accessToken,
  body: JSON.stringify({ serviceId, planId, pricingId }),
});
```

Retrieve invoice

```
const { invoice } = await billingFetch(`/invoices/${invoiceId}`, {
  clientToken: accessToken,
});
```

© 2026 WHMS. Confidential.

SDK Components

The SDK exports four ready-to-use React components. Each component is self-contained — it fetches its own data and manages its own loading/error state.

PricingTable

Fetches all active plans and renders them as a responsive card grid. Each card shows the plan name, description, pricing (with a billing-cycle selector when multiple cycles are available), and a CTA button.

```
import { PricingTable } from "@whms/billing-sdk";

<PricingTable
  apiKey="pk_test_xxx"
  baseUrl="https://api.example.com"
  serviceId="svc_123"          // optional — filters to one service
  onSelect={(plan, pricing) => {
    // Called when the user clicks "Select plan"
    navigate(`/checkout?planId=${plan.id}&pricingId=${pricing.id}`);
  }}
  className="my-pricing-grid"  // optional — override root CSS class
/>
```

Props

PROP	TYPE	REQUIRED	DESCRIPTION
`apiKey`	string	Yes	Public API key
`baseUrl`	string	Yes	Base URL of your billing API
`serviceId`	string	No	Filter plans to a specific service
`onSelect`	`(plan, pricing) => void`	No	Called on plan selection
`className`	string	No	CSS class for the root element

OrderForm

Handles the full checkout flow in one component: register a new client account (or silently skip if the email already exists), log in to obtain a client token, then create the order.

```
import { OrderForm } from "@whms/billing-sdk";

<OrderForm
  apiKey="pk_test_xxx"
  baseUrl="https://api.example.com"
  serviceId="svc_123"
  planId="plan_abc"
  pricingId="price_xyz"
  onSuccess={({ order, costBreakdown, accessToken }) => {
    // Redirect to a confirmation page, show a thank-you modal, etc.
    router.push(`/orders/${order.id}/confirmation`);
  }}
/>
```

Props

PROP	TYPE	REQUIRED	DESCRIPTION
`apiKey`	string	Yes	Public API key
`baseUrl`	string	Yes	Base URL of your billing API
`serviceId`	string	Yes	Target service
`planId`	string	Yes	Selected plan
`pricingId`	string	Yes	Selected pricing option
`onSuccess`	`({ order, costBreakdown, accessToken }) => void`	No	Called after successful order

LoginWidget

A minimal email/password form. On successful login, the access token is persisted in `localStorage` via the `useAuth` hook. When the user is already logged in, the widget displays their email and a logout button.

```
import { LoginWidget } from "@whms/billing-sdk";

<LoginWidget
  apiKey="pk_test_xxx"
  baseUrl="https://api.example.com"
  onLogin={({ user, accessToken }) => {
    // Optionally sync the token into your own state manager
    dispatch({ type: "SET_AUTH", payload: { user, accessToken } });
  }}
/>
```

Props

PROP	TYPE	REQUIRED	DESCRIPTION
`apiKey`	string	Yes	Public API key
`baseUrl`	string	Yes	Base URL of your billing API
`onLogin`	`({ user, accessToken, refreshToken }) => void`	No	Called after successful login
`className`	string	No	CSS class for the root element

InvoiceViewer

Displays a formatted invoice: number, status badge, issued/due dates, itemised line-item table, and a totals summary with tax and discounts.

```
import { InvoiceViewer } from "@whms/billing-sdk";

<InvoiceViewer
  apiKey="pk_test_xxx"
  baseUrl="https://api.example.com"
  invoiceId="inv_abc123"
  clientToken={accessToken}  // JWT from login — required for ownership check
/>
```

© 2026 WHMS. Confidential.

Props

PROP	TYPE	REQUIRED	DESCRIPTION
`apiKey`	string	Yes	Public API key
`baseUrl`	string	Yes	Base URL of your billing API
`invoiceId`	string	Yes	Invoice ID to display
`clientToken`	string	Yes	Client JWT from login
`className`	string	No	CSS class for the root element

© 2026 WHMS. Confidential.

SDK Hooks

For custom UI, import the hooks directly instead of the pre-built components.

usePlans

```
import { usePlans } from "@whms/billing-sdk";

const { plans, loading, error, refetch } = usePlans({
  apiKey:    "pk_test_xxx",
  baseUrl:   "https://api.example.com",
  serviceId: "svc_123",  // optional
});
```

Returns

FIELD	TYPE	DESCRIPTION
`plans`	`Plan[]`	Array of active plans (empty while loading)
`loading`	boolean	`true` while the request is in flight
`error`	`string | null`	Error message, or `null` on success
`refetch`	`() ⇒ void`	Manually re-trigger the fetch

useAuth

© 2026 WHMS. Confidential.

```
import { useAuth } from "@whms/billing-sdk";

const { user, token, loading, error, login, logout, register } = useAuth({
  apiKey:  "pk_test_xxx",
  baseUrl: "https://api.example.com",
});

// Login
const result = await login("user@example.com", "password123");
// → result: { user, accessToken, refreshToken }

// Register
const client = await register("new@example.com", "password123");

// Logout (clears localStorage)
logout();
```

Returns

FIELD	TYPE	DESCRIPTION
`user`	`Client | null`	Currently authenticated user
`token`	`string | null`	Access token (also in `localStorage`)
`loading`	boolean	`true` during async operations
`error`	`string | null`	Last error message
`login`	`async (email, password) ⇒ AuthResult`	Authenticate a client
`logout`	`() ⇒ void`	Clear session
`register`	`async (email, password) ⇒ Client`	Create a new account

© 2026 WHMS. Confidential.

End-to-End Integration Flow

A typical integration from zero to accepting orders takes five steps.

```
Step 1 ── Install SDK ──────────────────────────────────────┐
                                                            |
  npm install @whms/billing-sdk                             |
  # or load dist/widget.js via <script>                      |
                                                            |
Step 2 ── Create API key ──────────────────────────────────  |
                                                            |
  Admin Portal → API Keys → New Key                        |
  Scopes: plans.read  clients.create  auth.login            |
          orders.create  invoices.read                      |
                                                            |
Step 3 ── Embed pricing table ────────────────────────────  |
                                                            |
  <PricingTable                                             |
    apiKey="pk_live_xxx"                                    |
    baseUrl="https://api.example.com"                       |
    onSelect={(plan, pricing) => setSelected({ plan,        |
      pricing })}                                           |
  />                                                        |
                                                            |
Step 4 ── Accept orders ──────────────────────────────────  |
                                                            |
  When a plan is selected, render <OrderForm> with          |
  the planId and pricingId. The component handles           |
  registration, login, and order creation automatically.    |
                                                            |
Step 5 ── Manage clients ─────────────────────────────────  |
                                                            |
  Clients appear instantly in the Admin Portal.             |
  Orders, invoices, and provisioning are all                |
  handled automatically.                                    |
└───────────────────────────────────────────────────────────┘
```

© 2026 WHMS. Confidential.

Full React page example

```
"use client";  // Next.js App Router only
import { useState } from "react";
import { PricingTable, OrderForm } from "@whms/billing-sdk";

const API_KEY = "pk_live_xxxxxxxxxxxx";
const BASE    = "https://api.example.com";

export default function GetStartedPage() {
  const [selected, setSelected] = useState(null);

  if (selected) {
    return (
      <OrderForm
        apiKey={API_KEY}
        baseUrl={BASE}
        serviceId={selected.plan.service.id}
        planId={selected.plan.id}
        pricingId={selected.pricing.id}
        onSuccess={({ order }) => alert(`Order #${order.id} created!`)}
      />
    );
  }

  return (
    <PricingTable
      apiKey={API_KEY}
      baseUrl={BASE}
      onSelect={setSelected}
    />
  );
}
```

© 2026 WHMS. Confidential.

Security Best Practices

The golden rule

> **Never include your secret key (`sk_`) in any client-side code, bundle, or repository.**

Your public key (`pk_`) is designed to be embedded in frontend code. Your secret key must only ever exist on a backend server you control.

Key management checklist

```
✅ Use pk_ keys in all frontend / SDK / widget code
✅ Restrict pk_ keys to the minimum required scopes
✅ Set an expiry date on all API keys
✅ Rotate keys immediately if you suspect exposure
✅ Store sk_ keys in environment variables only
✅ Use separate keys for development and production

❌ Do NOT commit API keys to version control
❌ Do NOT share sk_ keys in Slack, email, or tickets
❌ Do NOT include sk_ keys in client-side bundles
❌ Do NOT log raw API keys in server logs
```

Environment variable pattern

```
# .env (never commit this file)
BILLING_API_KEY=pk_live_xxxxxxxxxxxx
BILLING_BASE_URL=https://api.example.com
```

```
// Safe — pk_ key read from environment at build time
<PricingTable
  apiKey={process.env.NEXT_PUBLIC_BILLING_API_KEY}
  baseUrl={process.env.NEXT_PUBLIC_BILLING_BASE_URL}
/>
```

© 2026 WHMS. Confidential.

CORS

The billing API allows requests from configured origin URLs only. Add your website domain to the allowed origins list in the Admin Portal → Settings → CORS. If you see CORS errors during development, add `http://localhost:3000` (or your local port) to the allowed list.

Content Security Policy

If your site uses a CSP header, whitelist the billing API domain:

```
Content-Security-Policy: connect-src 'self' https://api.example.com;
```

© 2026 WHMS. Confidential.

SDK Package Structure

```
packages/billing-sdk/
|
├── build.js                    # esbuild build script
├── package.json
|
├── src/
|   ├── index.js                # Library entry — named exports
|   ├── embed.js                # Widget bundle entry (window.BillingWidget)
|   |
|   ├── lib/
|   |   └── createClient.js     # Core API factory function
|   |
|   ├── hooks/
|   |   ├── usePlans.js         # Fetch & cache active plans
|   |   └── useAuth.js          # Login / logout / register + localStorage
|   |
|   └── components/
|       ├── PricingTable.jsx    # Plan card grid
|       ├── OrderForm.jsx       # Registration + checkout flow
|       ├── LoginWidget.jsx     # Email/password login form
|       └── InvoiceViewer.jsx   # Invoice detail + line items
|
└── dist/                       # Generated by npm run build
    ├── index.js                # CJS  (require())
    ├── index.mjs               # ESM  (import)
    └── widget.js               # IIFE (window.BillingWidget)
```

Build output summary

FILE	FORMAT	REACT BUNDLED	USE CASE
`dist/index.mjs`	ESM	No (peer dep)	Modern bundlers, Vite, Next.js
`dist/index.js`	CJS	No (peer dep)	Node.js, older bundlers
`dist/widget.js`	IIFE	**Yes**	`<script>` tag, no bundler needed

© 2026 WHMS. Confidential.

Building

```
cd packages/billing-sdk
npm install
npm run build
```

© 2026 WHMS. Confidential.

Use Case — Hosting Company

This section walks through a complete real-world scenario: a hosting company adding a fully functional billing flow to their existing marketing website.

Scenario

> **AcmeHost** runs a static marketing website (`acmehost.com`) built with plain HTML and Bootstrap. They want to add a pricing page that shows their hosting plans and lets visitors order directly — without rebuilding their site or adopting a new platform.

Step 1 — Generate a public API key

The AcmeHost admin logs into the billing portal and creates a new API key named `"Website Widget"` with scopes:

```
plans.read   clients.create   auth.login   orders.create   invoices.read
```

Step 2 — Add the widget to the pricing page

```
<!— acmehost.com/pricing.html —>
<link rel="stylesheet" href="/assets/bootstrap.min.css">

<section class="container py-5">
  <h2 class="text-center mb-4">Choose your plan</h2>
  <div id="acmehost-pricing"></div>
</section>

<!— Load the widget bundle —>
<script src="https://cdn.example.com/widget.js"></script>
<script>
  var selectedPlan = null;

  BillingWidget.renderPricingTable({
    elementId: "acmehost-pricing",
    apiKey:    "pk_live_xxxxxxxxxxxx",
    baseUrl:   "https://billing.acmehost.com",
    onSelect:  function(plan, pricing) {
      selectedPlan = { plan: plan, pricing: pricing };
      document.getElementById("order-section").style.display = "block";
      document.getElementById("order-section").scrollIntoView();

      // Re-render the order form with the selected plan
      BillingWidget.renderOrderForm({
        elementId: "acmehost-order",
        apiKey:    "pk_live_xxxxxxxxxxxx",
        baseUrl:   "https://billing.acmehost.com",
        serviceId: plan.service.id,
        planId:    plan.id,
        pricingId: pricing.id,
        onSuccess: function(result) {
          window.location.href = "/thank-you.html?order=" + result.order.id;
        }
      });
    }
  });
</script>
```

```
<!— Order form — hidden until a plan is selected —>
<section id="order-section" class="container py-5" style="display:none">
  <h2 class="text-center mb-4">Complete your order</h2>
  <div id="acmehost-order" class="mx-auto" style="max-width:400px"></div>
</section>
```

Step 3 — Go live

AcmeHost deploys `pricing.html`. Within minutes:

- Visitors see live plans pulled directly from the billing engine
- New customers register and place orders through the embedded form
- Orders appear instantly in the admin portal
- Invoices are generated and sent automatically
- Provisioning workflows trigger on order confirmation

No backend code was written. No database was touched. Total integration time: under 30 minutes.

Troubleshooting

`401 — API key required` or `Invalid or expired API key`

- Verify the key is being sent as `x-api-key: <your-key>` or `Authorization: Bearer <your-key>`
- Confirm the key has not expired — check the Admin Portal → API Keys
- Confirm the key has not been revoked
- For production keys, confirm you are using the `pk_live_` prefix, not `pk_test_`

`403 — Insufficient scope`

Your API key does not have the required scope for this endpoint.

```
Required: orders.create
Your key scopes: plans.read, auth.login
```

Go to Admin Portal → API Keys → edit the key and add the missing scope.

`403 — Access denied` on invoices

The `x-client-token` you provided does not belong to the owner of this invoice. Ensure you are passing the token returned from the login call made by the correct user.

CORS errors in the browser

```
Access to fetch at 'https://api.example.com' from origin 'https://mysite.com'
has been blocked by CORS policy
```

Add your site's origin to the allowed list:

- Admin Portal → Settings → CORS Origins → Add `https://mysite.com`
- In development, add `http://localhost:3000`

`TypeError: Failed to fetch` / network errors

- Confirm the `baseUrl` does not have a trailing slash: `https://api.example.com` ✅ not `https://api.example.com/` ❌

- Check that the billing API server is running and reachable
- Verify firewall / reverse proxy rules are not blocking `/public/v1/*` routes

---

Plans not showing (`plans: []`)

- Confirm at least one plan is marked **active** in Admin Portal → Services → Plans
- If using `serviceId`, confirm the ID is correct and the service is published
- Check that the API key has the `plans.read` scope

`409 — Email already registered`

The `POST /public/v1/clients` endpoint returned a conflict. The `OrderForm` component handles this automatically by attempting a login instead. If you are calling the endpoint manually, catch the `409` and proceed to login:

```
try {
  await billingFetch("/clients", { method: "POST", body: JSON.stringify({ email, password })
} catch (err) {
  if (!err.message.includes("already registered")) throw err;
  // continue to login
}
```

Widget does not render (script tag)

- Confirm the `elementId` matches an element that exists in the DOM **before** the script runs
- Move the `<script>` tag to just before `</body>`, or wrap the render call in `DOMContentLoaded`:

```
<script>
  document.addEventListener("DOMContentLoaded", function() {
    BillingWidget.renderPricingTable({ elementId: "pricing", ... });
  });
</script>
```

For further support, contact the platform team or open an issue in the internal repository.

© 2026 WHMS. Confidential.